

Report 2: Q33

Maximum Product Sub array Using Dynamic Programming

Aim

To develop a program that finds the contiguous subarray having the maximum product from a given array of integers using Dynamic Programming.

Objective

The objective of this experiment is to understand the Dynamic Programming approach for solving optimization problems involving arrays. The problem is challenging because the array may contain negative numbers, which can change the sign of the product and affect the final result.

Problem Description

Given an array of integers, the task is to find the contiguous subarray that produces the maximum possible product. Unlike the Maximum Sum Subarray problem, the presence of negative numbers makes the problem more complex because multiplying two negative numbers results in a positive value.

For example:

Array = [2, 3, -2, 4, -1]

Product of the entire array:

$$2 \times 3 \times (-2) \times 4 \times (-1) = 48$$

Hence, the maximum product obtained is 48.

Dynamic Programming Concept

At each position, two values are maintained:

1. Maximum product ending at the current position.
2. Minimum product ending at the current position.

The minimum product is stored because multiplying it by a negative number may produce a larger positive product in subsequent steps.

This approach avoids recalculating products repeatedly and improves efficiency.

Algorithm

1. Initialize maximum product, minimum product, and result with the first element.
2. Traverse the array from the second element.
3. If the current element is negative, swap maximum and minimum products.
4. Update maximum product.
5. Update minimum product.
6. Update the overall maximum product.
7. Continue until the array is completely processed.
8. Display the maximum product.

Python Program

```
def max_product_subarray(arr):  
    max_ending_here = arr[0]  
    min_ending_here = arr[0]  
    max_so_far = arr[0]  
  
    for i in range(1, len(arr)):  
        current = arr[i]
```

```

if current < 0:
    max_ending_here, min_ending_here = min_ending_here, max_ending_here

max_ending_here = max(current, max_ending_here * current)
min_ending_here = min(current, min_ending_here * current)

max_so_far = max(max_so_far, max_ending_here)

return max_so_far

n = int(input("Enter number of elements: "))
arr = list(map(int, input().split()))

print("Max Product =", max_product_subarray(arr))

```

Sample Input

n = 5

arr = 2 3 -2 4 -1

Sample Output

Max Product = 48

Dry Run

Array = [2, 3, -2, 4, -1]

Step	1:
Max = 2, Min = 2, Result = 2	
Step	2:
Element = 3	

Max = 6

Min = 3

Result = 6

Step

3:

Element = -2

Max = -2

Min = -12

Result = 6

Step

4:

Element = 4

Max = 4

Min = -48

Result = 6

Step

5:

Element = -1

Max = 48

Min = -4

Result = 48

Final Answer = 48

Analysis

The Dynamic Programming approach efficiently handles positive and negative numbers by maintaining both maximum and minimum products at every step. This ensures that any future multiplication involving a negative number is properly considered. The algorithm processes each element only once, making it highly efficient for large datasets.

Advantages

- Efficient and optimized solution.
- Handles negative numbers correctly.
- Requires constant extra memory.
- Suitable for large input arrays.

Applications

- Financial data analysis.
- Signal processing.
- Data mining and pattern recognition.
- Machine learning preprocessing.
- Scientific and numerical computations.

Time Complexity

Array Traversal: $O(n)$

Overall Time Complexity: $O(n)$

Space Complexity

$O(1)$

Result

The Maximum Product Subarray problem was successfully solved using Dynamic Programming. By maintaining both maximum and minimum products at each position, the algorithm efficiently finds the maximum possible product of any contiguous subarray. The obtained result for the sample input is 48, validating the correctness and effectiveness of the solution.